Document parsers struggle with extracting complex layouts common in real-world documents, such as tables having merged cells, cross-page breaks, and misplaced text. Moreover, a large part of the information lies in charts/graphs or figures, which require accurate extraction. This article walks you through a parser that extracts all multimodal data from documents with accurate layout preservation. It demonstrates how you can use it as a reusable Python package that works with multiple model providers.

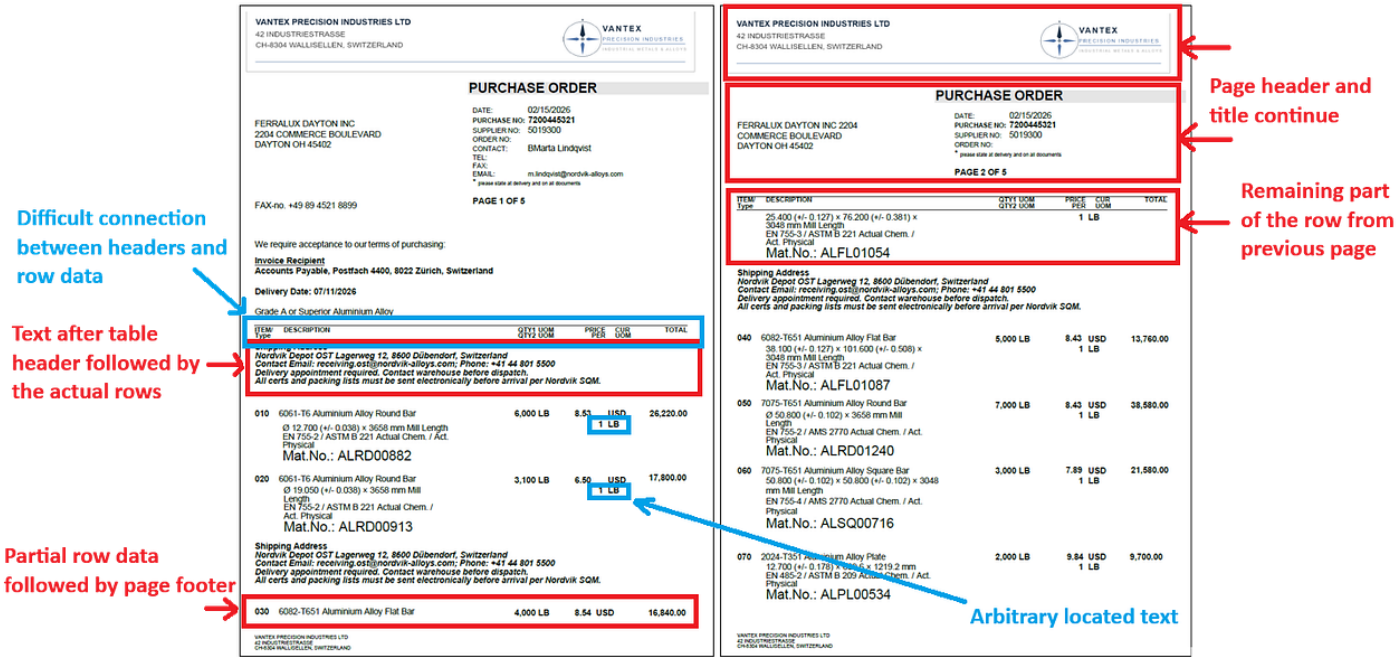
Accurate extraction of content from documents is critical for AI applications. For instance, while processing a **purchase order for generating a sales draft using AI**, an AI agent or an LLM first needs to accurately understand the purchase order content with the exact layout.

A misaligned table header can make the LLM read the wrong quantity, which will result in wrong price calculation. Similarly, a missed line item due to a cross-page table break can cause the LLM to skip a product entirely.

And we have seen this happening quite often in our **generative AI project** (GAIK), in which we build AI applications with enterprise documents, using our **open-source generative AI toolkit** (see GAIK's GitHub repository here).

Many customer documents, such as purchase orders or bills of materials, contain messy tables. As an example, see the following 2 pages of a purchase order (content changed for privacy reasons while preserving the layout).

Press enter or click to view image in full size



There is a text between the table header and row data. Of course, this is not the right place to add this text. It could have been added to some other place before or after the table. But it is quite common to expect such structures from customer documents. This text is also unnecessarily repeated in other places in the same table.

The table continues on multiple pages with row data partially written across pages. For instance, the first page contains partial data for the third item (030), followed by the table footer, and page header and title (repeated) on the next page.

Specialized parsers, such as **PyMuPDF, PyMuPDF4LLM**, or **Docling**, do not accurately extract this table structure.

I addressed this issue in my following articles:

How to Extract Everything from Documents Using AI

I created a vision-enhanced document parser and RAG chunker

medium.com

Building a Generic Knowledge Extraction AI Framework for Organization-Specific Use Cases

Allowing non-technical users to specify requirements in plain language, automatically generate schemas, and extract...

medium.com

Even these two parsers do not correctly extract such messy table layouts. See the following extraction snapshots.

Press enter or click to view image in full size

020 6061-T6 Aluminium Alloy Round Bar
Ø 19.050 (+/- 0.038) x 3658 mm Mill Length
EN 755-2 / ASTM B 221 Actual Chem. / Act. Physical
3 , 1 00 LB
6 .50
1 LB
USD
1 7 , 8 00.00
Mat.No.: ALRD00913

Shipping Address
Nordvik Depot OST Lagerweg 12, 8600 Dübendorf, Switzerland Contact Email: receiving.ost@nordvik-alloys.com; Phone: +41 44 801 5500 D
warehouse before dispatch.

All certs and packing lists must be sent electronically before arrival per Nordvik SQM.
030 6082-T651 Aluminium Alloy Flat Bar
4 ,000 LB
8 . 54 USD
1 6 , 8 40.00

← Partial data of third item on first page

FERRALOX DAYTON INC 2204 COMMERCE BOULEVARD
DAYTON OH 45402
<!-- image -->
PURCHASE ORDER
DATE:
02/15/2026
PURCHASE NO:
7200445321
SUPPLIER NO: ORDER NO:
6019300
* please state at delivery and on all documents
PAGE 2 OF 5

← Header/footer between the data of item 030

ITEM/ DESCRIPTION
Type
QTY1
QTY2
UOM
UOM
PRICE
PER
CUR
UOM
TOTAL
25.400 (+/- 0.127) x 76.200 (+/- 0.381) x 3048 mm Mill Length EN 755-3 / ASTM B 221 Actual Chem. / Act. Physical Mat.No.: ALFL01054
1 LB

Remaining data of third item (030) on second page

Shipping Address

Partial view of the table extracted by Docling+LLM parser (image by author)

Press enter or click to view image in full size

Invoice Recipient
Accounts Payable, Postfach 4400, 8022 Zürich, Switzerland

Delivery Date: 07/11/2026

Grade A or Superior Aluminium Alloy

ITEM/Type	DESCRIPTION	QTY1 UOM QTY2 UOM	PRICE PER	CUR UOM	TOTAL
	Shipping Address **Nordvik Depot OST Lagerweg 12, 8600 Dübendorf, Switzerland** **Contact Email: receiving.ost@nordvik-alloys.com; Phone: +41 44 801 5500** **Delivery appointment required. Contact warehouse before dispatch.** **All certs and packing lists must be sent electronically before arrival per Nordvik SQM.**				
010	6061-T6 Aluminium Alloy Round Bar Ø 12.700 (+/- 0.038) x 3658 mm Mill Length EN 755-2 / ASTM B 221 Actual Chem. / Act. Physical Mat.No.: ALRD00882	**6,000 LB**	**8.53**	**USD** **1 LB**	**26,220.00**
020	6061-T6 Aluminium Alloy Round Bar Ø 19.050 (+/- 0.038) x 3658 mm Mill Length EN 755-2 / ASTM B 221 Actual Chem. / Act. Physical Mat.No.: ALRD00913	**3,100 LB**	**6.50**	**USD** **1 LB**	**17,800.00**
	Shipping Address **Nordvik Depot OST Lagerweg 12, 8600 Dübendorf, Switzerland** **Contact Email: receiving.ost@nordvik-alloys.com; Phone: +41 44 801 5500** **Delivery appointment required. Contact warehouse before dispatch.** **All certs and packing lists must be sent electronically before arrival per Nordvik SQM.**				
030	6082-T651 Aluminium Alloy Flat Bar 25.400 (+/- 0.127) x 76.200 (+/- 0.381) x 3048 mm Mill Length EN 755-3 / ASTM B 221 Actual Chem. / Act. Physical Mat.No.: ALFL01054 Shipping Address Nordvik Depot OST Lagerweg 12, 8600 Dübendorf, Switzerland Contact Email: receiving.ost@nordvik-alloys.com; Phone: +41 44 801 5500 Delivery appointment required. Contact warehouse before dispatch. All certs and packing lists must be sent electronically before arrival per Nordvik SQM.	**4,000 LB**	**8.54**	**USD** **1 LB**	**16,840.00**
040	6082-T651 Aluminium Alloy Flat Bar 38.100 (+/- 0.127) x 101.600 (+/- 0.508) x 3048 mm Mill Length EN 755-3 / ASTM B 221 Actual Chem. / Act. Physical Mat.No.: ALFL01087	5,000 LB	8.43	USD 1 LB	13,760.00
050	7075-T651 Aluminium Alloy Round Bar Ø 50.800 (+/- 0.102) x 3658 mm Mill Length EN 755-2 / AMS 2770 Actual Chem. / Act. Physical Mat.No.: ALRD01240	7,000 LB	8.43	USD 1 LB	38,580.00
060	7075-T651 Aluminium Alloy Square Bar 50.800 (+/- 0.102) x 50.800 (+/- 0.102) x 3048 mm Mill Length EN 755-4 / AMS 2770 Actual Chem. / Act. Physical Mat.No.: ALSQ00716	3,000 LB	7.89	USD 1 LB	21,580.00
070	2024-T351 Aluminium Alloy Plate 12.700 (+/- 0.178) x 609.6 x 1219.2 mm EN 485-2 / ASTM B 209 Actual Chem. / Act. Physical Mat.No.: ALPL00534	2,000 LB	9.84	USD 1 LB	9,700.00

VANTEX PRECISION INDUSTRIES LTD

Partial view of the table extracted by GAIK's Vision+ parser using vision LLM (image by author)

The fix ultimately turned out to be much simpler than I expected.

It was all about using the right prompts. Just a different way of asking the AI models what you want.

Earlier this month, LlamaIndex published the document parsing prompts they used in their benchmark study, and those prompts changed everything.

Here's the proof. The same purchase order that failed every other parser we tried:

ITEM/ Type	DESCRIPTION	QTY1 UOM QTY2 UOM	PRICE PER	CUR UOM	TOTAL
Shipping Address Nordvik Depot OST Lagerweg 12, 8600 Dübendorf, Switzerland Contact Email: receiving.ost@nordvik-alloys.com; Phone: +41 44 801 5500 Delivery appointment required. Contact warehouse before dispatch. All certs and packing lists must be sent electronically before arrival per Nordvik SQM.					
010	6061-T6 Aluminium Alloy Round Bar Ø 12.700 (+/- 0.038) × 3658 mm Mill Length EN 755-2 / ASTM B 221 Actual Chem. / Act. Physical Mat.No.: ALRD00882	6,000 LB	8.53 1	USD LB	26,220.00
020	6061-T6 Aluminium Alloy Round Bar Ø 19.050 (+/- 0.038) × 3658 mm Mill Length EN 755-2 / ASTM B 221 Actual Chem. / Act. Physical Mat.No.: ALRD00913	3,100 LB	6.50 1	USD LB	17,800.00
Shipping Address Nordvik Depot OST Lagerweg 12, 8600 Dübendorf, Switzerland Contact Email: receiving.ost@nordvik-alloys.com; Phone: +41 44 801 5500 Delivery appointment required. Contact warehouse before dispatch. All certs and packing lists must be sent electronically before arrival per Nordvik SQM.					
030	6082-T651 Aluminium Alloy Flat Bar 25.400 (+/- 0.127) × 76.200 (+/- 0.381) × 3048 mm Mill Length EN 755-3 / ASTM B 221 Actual Chem. / Act. Physical Mat.No.: ALFL01054	4,000 LB	8.54 1	USD LB	16,840.00
Shipping Address Nordvik Depot OST Lagerweg 12, 8600 Dübendorf, Switzerland Contact Email: receiving.ost@nordvik-alloys.com; Phone: +41 44 801 5500 Delivery appointment required. Contact warehouse before dispatch.					

Partial view (converted to HTML for better readability) of the purchase order table extracted by GAIK's multimodal parser with gemini-3.1-flash-lite-preview model with reasoning effort set to low (image by author)

In this article, we will create a multimodal parser to accurately extract/parse the content from documents with messy layouts.

The multimodal parser allows the user to select a provider and a model (**OpenAI, Anthropic, or Google**) with several other options to evaluate parsing quality for their use case.

The source code of the multimodal parser with detailed guidelines and documentation can be found at the following link in the GAIK toolkit's GitHub repository: **multimodal_parser**. It requires API credentials for at least one provider (**OpenAI, Azure, or Google Vertex AI**).

It Was All About Some Specific Prompts

Earlier this month, **LlamaIndex** launched an open-source framework (**ParseBench**) for benchmarking how well document parsers convert PDFs into output that AI agents can actually use. They tested 14 different methods: general-purpose vision LLMs, specialized parsers, and their own LlamaParse.

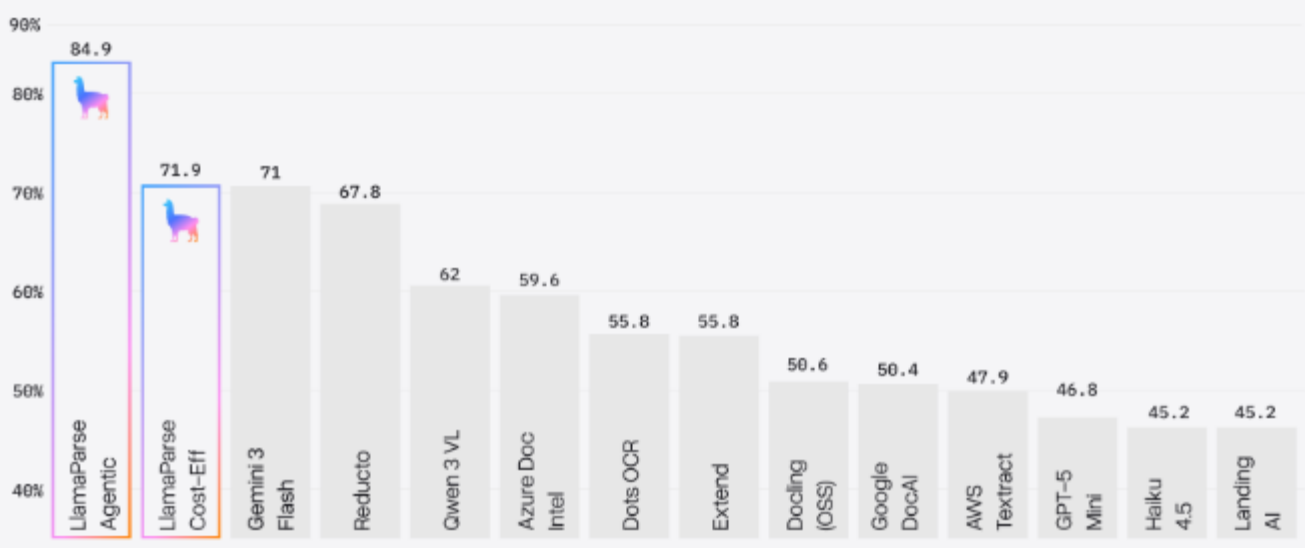
Their benchmark results are published in the following study:

ParseBench: A Document Parsing Benchmark for AI Agents

AI agents are changing the requirements for document parsing. What matters is semantic correctness: parsed output must…

arxiv.org

These results show that the agentic parsing (such as their own LlamaParse) and LLM-based parsing (such as with Gemini 3 Flash) outperform specialized document parsers.



Agentic and VLM-based parsing outperform specialized parsers. Image source: https://www.llamaindex.ai/blog/parsebench

I was curious to know how they got the LLM-based parsing to work so well. So I went through the actual research paper.

I expected some clever architectural trick. A fine-tuned model, or a multi-step pipeline with verification loops.

Instead, I found some specific prompts. The prompts that we had never thought to use.

What is Special About These Prompts?

When you ask an LLM to output a table as Markdown (the default for most parsers), it doesn't preserve the layout of merged cells and multi-level headers correctly. For instance, a purchase order with a two-row header that spans multiple columns becomes flat and ambiguous.

LlamaIndex's proposed prompts extract a table with HTML colspan and rowspan attributes, which encode the full structure. The prompts ask the LLM to convert tables to HTML format (<table>, <tr>, <th>, <td>), and use colspan and rowspan attributes to preserve merged cells and hierarchical headers.

Similarly, for charts or graphs, the LLM is asked to convert them to tables using flat combined column headers so that the row of each data cell contains all its labels.

The paper proposes to use a shared system prompt for OpenAI and Claude models. For Google models, the user prompt is a bit different.

Here are LlamaParse's proposed system and user prompts for OpenAI and Claude models that we will be using for building a reusable multimodal parser package.

OPENAI_CLAUDE_SYSTEM_PROMPT = """You are a document parser. Your task is to convert document PDFs into clean, well-structured Markdown.

Guidelines:
- Preserve the document structure, including headings, paragraphs, lists, and tables.
- Convert tables to HTML using `<table>`, `<tr>`, `<th>`, and `<td>`.
- For existing tables in the document, use `colspan` and `rowspan` attributes to preserve merged cells and hierarchical headers.
- For charts or graphs converted into tables, use flat combined column headers (for example, "Primary 2015" instead of separate header rows) so that each data cell's row contains all of its labels.
- Describe images and figures briefly in square brackets, for example: `[Figure: description]`.
- Preserve any code blocks with appropriate syntax highlighting.
- Maintain reading order: left to right, top to bottom for Western documents.
- Do not add commentary or explanations. Output only the parsed content.

Additionally, wrap each layout element in a `<div>` tag with:
- `data-bbox="[x1, y1, x2, y2]"` for the bounding box in normalized 0-1000 coordinates, where x is horizontal (left edge = 0, right edge = 1000) and y is vertical (top = 0, bottom = 1000). `x1, y1` is the top-left corner and `x2, y2` is the bottom-right corner.
- `data-label="<category>"` where category is one of: `Caption`, `Footnote`, `Formula`, `List-item`, `Page-footer`, `Page-header`, `Picture`, `Section-header`, `Table`, `Text`, `Title`.

Place elements in reading order. Every piece of content must be inside exactly one `<div>` wrapper."""

OPENAI_CLAUDE_USER_PROMPT = """The attached PDF is read from the input folder next to this script.

Parse the full document and output its content as clean markdown, with each layout element wrapped in a tag. Use HTML tables for any tabular data. For charts and graphs, use flat combined column headers. Output ONLY the parsed content with div wrappers and no explanations.
"""

Google's Gemini models use the same prompts, with the only difference that the data-bbox format is changed to their native coordinate order.

For Gemini models, the following system and user prompts are used:

GOOGLE_SYSTEM_PROMPT = """You are a document parser. Your task is to convert document PDFs into clean, well-structured Markdown.

Guidelines:
- Preserve the document structure, including headings, paragraphs, lists, and tables.
- Convert tables to HTML using `<table>`, `<tr>`, `<th>`, and `<td>`.
- For existing tables in the document, use `colspan` and `rowspan` attributes to preserve merged cells and hierarchical headers.
- For charts or graphs converted into tables, use flat combined column headers (for example, "Primary 2015" instead of separate header rows) so that each data cell's row contains all of its labels.
- Describe images and figures briefly in square brackets, for example: `[Figure: description]`.
- Preserve any code blocks with appropriate syntax highlighting.
- Maintain reading order: left to right, top to bottom for Western documents.
- Do not add commentary or explanations. Output only the parsed content.

Additionally, wrap each layout element in a `<div>` tag with:
- `data-bbox="[y_min, x_min, y_max, x_max]"` for the bounding box in normalized 0-1000 coordinates where x is horizontal (left edge = 0, right edge = 1000) and y is vertical (top = 0, bottom = 1000). The order is `[y_min, x_min, y_max, x_max]`.
- `data-label="<category>"` where category is one of: `Caption`, `Footnote`, `Formula`, `List-item`, `Page-footer`, `Page-header`, `Picture`, `Section-header`, `Table`, `Text`, `Title`.

Place elements in reading order. Every piece of content must be inside exactly one `<div>` wrapper."""

GOOGLE_USER_PROMPT = """Parse this document page and output its content as clean markdown, with each layout element wrapped in a tag.
Use HTML tables for any tabular data. For charts/graphs, use flat combined column headers. Output ONLY the parsed content with div wrappers, no explanations.
"""

Instead of Markdown tables that do not accurately represent merged cells, and messy real-world tables, the prompt asks for HTML tables with colspan and rowspan. Every layout element gets a

bounding box in normalized coordinates. This means downstream code can reconstruct the spatial layout of the page without depending on the original PDF renderer.

The Google prompts use a different bounding-box coordinate order ([y_min, x_min, y_max, x_max]) because that matches what Gemini models produce natively.

The parsing prompts are defined in GitHub's prompts.py script.

Multimodal Parser for Accurate Parsing

I packaged a parsing pipeline using the above-mentioned prompts into a Python class that reads a PDF document and lets the user select a provider and model (**OpenAI, Anthropic, or Google**) with several other options.

The full code structure is in the **GitHub repository**. The complete implementation of the MultimodalParser class is in multimodal_parser.py.

The MultimodalParser has just one method, parse(), that orchestrates the whole pipeline.

```
parser = MultimodalParser(
  model_provider="google",
  model="gemini-3.1-flash-lite-preview",
  reasoning_effort="low",
  merge_table=True,
  create_html=True,
)
result = parser.parse("document.pdf")
```

A few parameters worth understanding:

- reasoning_effort — model's thinking budget: "low", "medium", or "high". Higher effort can improve accuracy on complex layouts, but is slower and costlier.
- merge_table — when True, appends an instruction to the user prompt telling the model to combine tables split across pages.
- additional_instructions — additional instructions appended to the user prompt for domain-specific rules.
- create_html — when True, renders the cleaned markdown into an HTML document.

The parse() method starts by reading the PDF as raw bytes and encoding it as a base64 string, so that it can be embedded in a JSON API payload and sent to the selected LLM with the aforementioned provider-specific prompts.

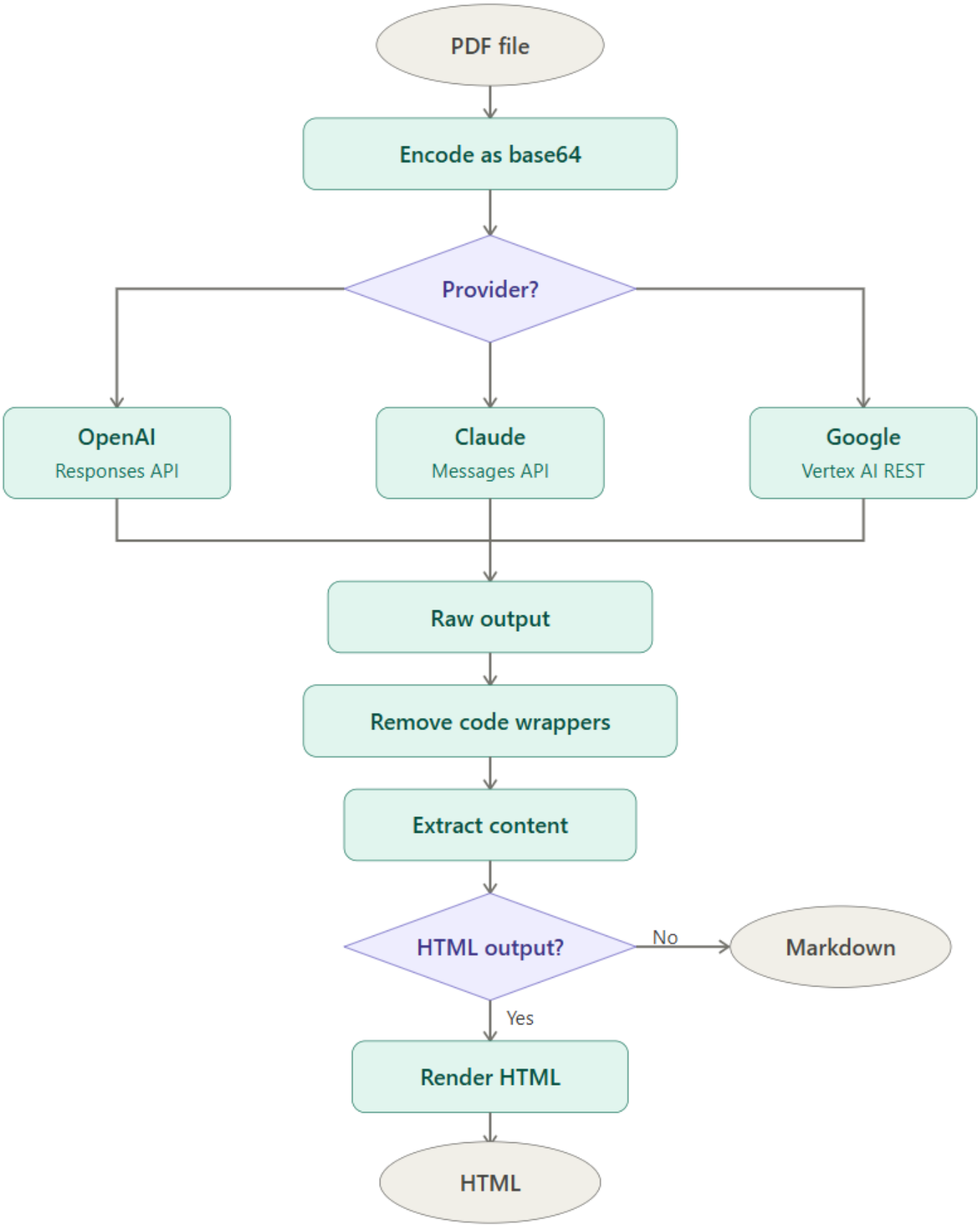
It then builds the right message structure based on the selected provider, because OpenAI, Claude, and Google each expect their content to be formatted differently.

After the LLM API call, the model returns its raw response, which is a combination of markdown text and wrappers containing bounding box coordinates for every layout element on the page. A cleanup process removes any code block fences that the model may have wrapped the output in. Another method extracts the actual content from those wrappers in the form of clean markdown.

If the option create_html=True, the clean markdown is converted into a styled HTML document that you can open directly in a browser to visually inspect the extraction.

Every call returns a UsageRecord alongside the parsed content, which provides the details of token consumption and the associated cost.

Press enter or click to view image in full size



The pipeline of the multimodal parser(image by author)

How to Use It

The multimodal parser has been created as a reusable software component of our GAIK project's open-source generative AI toolkit. This software component can be installed as a standalone Python package as follows:

pip install "gaik[multimodal-parser]"

Here is a minimal example using gemini-3.1-flash-lite-preview with low reasoning efforts to parse the same purchase order. This example saves the parsing output in raw markdown, clean markdown, and an HTML format.

See a detailed example with token consumption and price calculation at this link.

```
from pathlib import Path
from dotenv import load_dotenv
from gaik.software_components.parsers.multimodal_parser import MultimodalParser

load_dotenv()

OUTPUT_DIR = Path(__file__).parent / "output"

def save_result(result, prefix: str) -> None:
  """Save parse results to the output directory."""
  OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

  raw_path = OUTPUT_DIR / f"{prefix}_raw.md"
  raw_path.write_text(result.raw_markdown, encoding="utf-8")

  clean_path = OUTPUT_DIR / f"{prefix}_clean.md"
  clean_path.write_text(result.clean_markdown, encoding="utf-8")

  print(f"Saved: {raw_path}")
  print(f"Saved: {clean_path}")

  if result.html is not None:
    html_path = OUTPUT_DIR / f"{prefix}_clean.html"
    html_path.write_text(result.html, encoding="utf-8")
    print(f"Saved: {html_path}")

parser = MultimodalParser(
  model_provider="google",
  model="gemini-3.1-flash-lite-preview",
  reasoning_effort="low",
  merge_table=True,
  create_html=True,
```

)
result = parser.parse("sample_PO.pdf")
save_result(result, "output_google")

The output for the same purchase order, correctly merged and structured, is shown above (HTML format).

Here are a few more input-output examples.

Press enter or click to view image in full size

6. Direct and indirect family holdings of stock, by selected characteristics of families, 1989, 1992, 1995, and 1998 surveys

Percent except as noted

Family characteristic	Families having stock holdings, direct or indirect[1]				Median value among families with holdings (thousands of 1998 dollars)				Stock holdings as share of group's financial assets			
	1989	1992	1995	1998	1989	1992	1995	1998	1989	1992	1995	1998
All families	**31.6**	**36.7**	**40.4**	**48.8**	**10.8**	**12.0**	**15.4**	**25.0**	**27.8**	**33.7**	**40.0**	**53.9**
Income (1998 dollars)												
Less than 10,000	*	6.8	5.4	7.7	*	6.2	3.2	4.0	*	15.9	12.9	24.8
10,000–24,999	12.7	17.8	22.2	24.7	6.4	4.6	6.4	9.0	11.7	15.3	26.7	27.5
25,000–49,999	31.5	40.2	45.4	52.7	6.0	7.2	8.5	11.5	16.9	23.7	30.3	39.1
50,000–99,999	51.5	62.5	65.4	74.3	10.2	15.4	23.6	35.7	23.2	33.5	39.9	48.8
100,000 or more	81.8	78.3	81.6	91.0	53.5	71.9	85.5	150.0	35.3	40.2	46.4	63.0
Age of head (years)												
Less than 35	22.4	28.3	36.6	40.7	3.8	4.0	5.4	7.0	20.2	24.8	27.2	44.8
35–44	38.9	42.4	46.4	56.5	6.6	8.6	10.6	20.0	29.2	31.0	39.5	54.7
45–54	41.8	46.4	48.9	58.6	16.7	17.1	27.6	38.0	33.5	40.6	42.9	55.7
55–64	36.2	45.3	40.0	55.9	23.4	28.5	32.9	47.0	27.6	37.3	44.4	58.3
65–74	26.7	30.2	34.4	42.6	25.8	18.3	36.1	56.0	26.0	31.6	35.8	51.3
75 or more	25.9	25.7	27.9	29.4	31.8	28.5	21.2	60.0	25.0	25.4	39.8	48.7

NOTE. See note to table 1.
1. Indirect holdings are those in mutual funds, retirement accounts, and other managed assets.
* Ten or fewer observations.

A table with a complex layout (image by author)

Results from the 1998 Survey of Consumer Finances 15

6. Direct and indirect family holdings of stock, by selected characteristics of families, 1989, 1992, 1995, and 1998 surveys
Percent except as noted

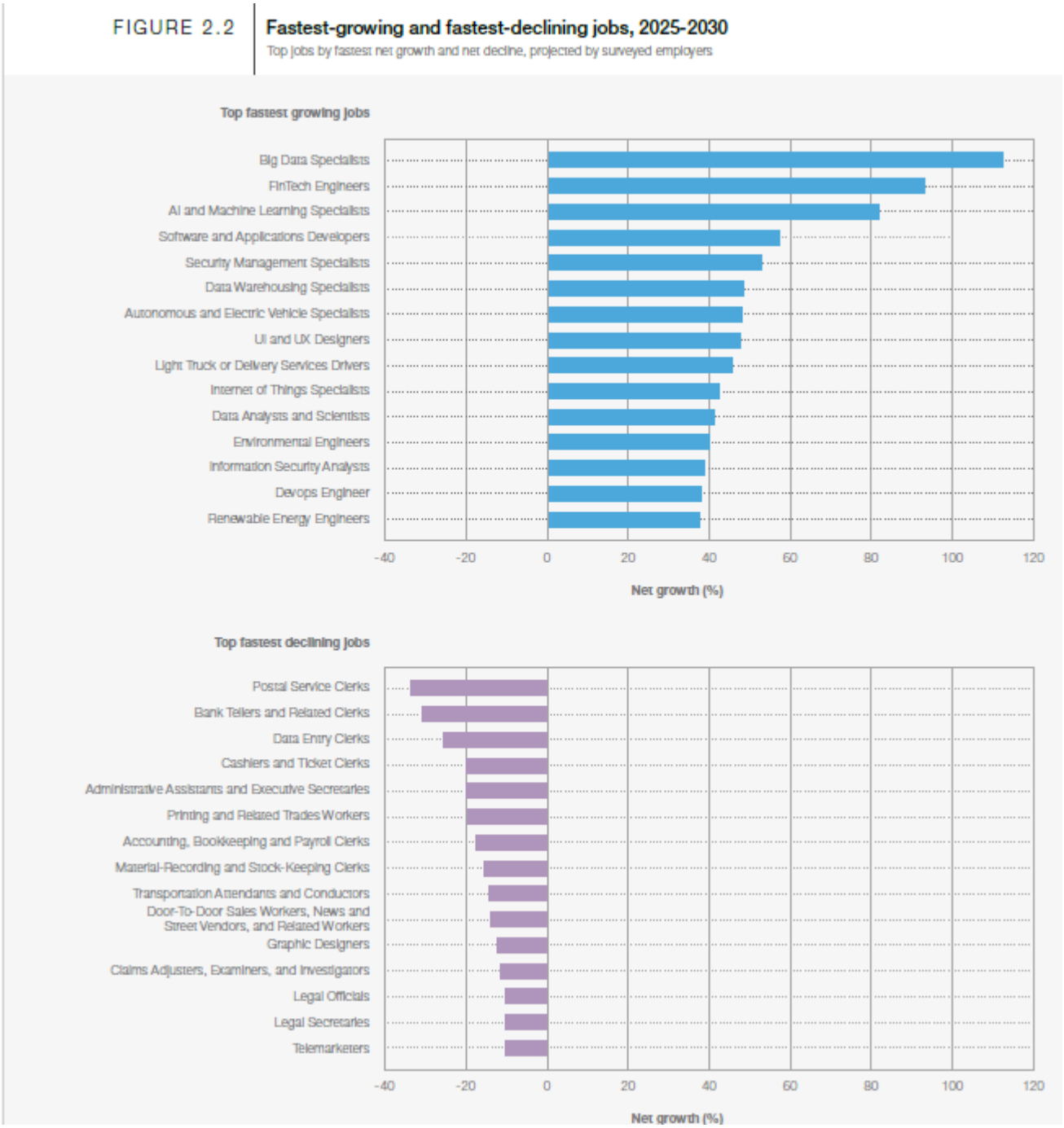
Family characteristic	Families having stock holdings, direct or indirect[1]				Median value among families with holdings (thousands of 1998 dollars)				Stock holdings as share of group's financial assets			
	1989	1992	1995	1998	1989	1992	1995	1998	1989	1992	1995	1998
All families	**31.6**	**36.7**	**40.4**	**48.8**	**10.8**	**12.0**	**15.4**	**25.0**	**27.8**	**33.7**	**40.0**	**53.9**
Income (1998 dollars)												
Less than 10,000	*	6.8	5.4	7.7	*	6.2	3.2	4.0	*	15.9	12.9	24.8
10,000-24,999	12.7	17.8	22.2	24.7	6.4	4.6	6.4	9.0	11.7	15.3	26.7	27.5
25,000-49,999	31.5	40.2	45.4	52.7	6.0	7.2	8.5	11.5	16.9	23.7	30.3	39.1
50,000-99,999	51.5	62.5	65.4	74.3	10.2	15.4	23.6	35.7	23.2	33.5	39.9	48.8
100,000 or more	81.8	78.3	81.6	91.0	53.5	71.9	85.5	150.0	35.3	40.2	46.4	63.0
Age of head (years)												
Less than 35	22.4	28.3	36.6	40.7	3.8	4.0	5.4	7.0	20.2	24.8	27.2	44.8
35-44	38.9	42.4	46.4	56.5	6.6	8.6	10.6	20.0	29.2	31.0	39.5	54.7
45-54	41.8	46.4	48.9	58.6	16.7	17.1	27.6	38.0	33.5	40.6	42.9	55.7
55-64	36.2	45.3	40.0	55.9	23.4	28.5	32.3	47.0	27.6	37.3	44.4	58.3
65-74	26.7	30.2	34.4	42.6	25.8	18.3	36.1	56.0	26.0	31.6	35.8	51.3
75 or more	25.9	25.7	27.9	29.4	31.8	28.5	21.2	60.0	25.0	25.4	39.8	48.7

NOTE. See note to table 1.

1. Indirect holdings are those in mutual funds, retirement accounts, and other managed assets.

* Ten or fewer observations.

The table parsed by gemini-3.1-flash-lite-preview with reasoning effort 'low' (image by author)



An example document having two charts (image by author)

FIGURE 2.2

Fastest-growing and fastest-declining jobs, 2025-2030

Top jobs by fastest net growth and net decline, projected by surveyed employers

Top fastest growing jobs

Job Role	Net growth (%)
Big Data Specialists	114
FinTech Engineers	94
AI and Machine Learning Specialists	82
Software and Applications Developers	57
Security Management Specialists	53
Data Warehousing Specialists	49
Autonomous and Electric Vehicle Specialists	48
UI and UX Designers	48
Light Truck or Delivery Services Drivers	47
Internet of Things Specialists	42
Data Analysts and Scientists	41
Environmental Engineers	40
Information Security Analysts	39
Devops Engineer	38
Renewable Energy Engineers	37

Top fastest declining jobs

Job Role	Net growth (%)
Postal Service Clerks	-31
Bank Tellers and Related Clerks	-28
Data Entry Clerks	-26
Cashiers and Ticket Clerks	-20
Administrative Assistants and Executive Secretaries	-20
Printing and Related Trades Workers	-19
Accounting, Bookkeeping and Payroll Clerks	-17
Material-Recording and Stock-Keeping Clerks	-16
Transportation Attendants and Conductors	-15
Door-To-Door Sales Workers, News and Street Vendors, and Related Workers	-14
Graphic Designers	-12
Claims Adjusters, Examiners, and Investigators	-11
Legal Officials	-10
Legal Secretaries	-10
Telemarketers	-10

The two charts parsed by gemini-3.1-flash-lite-preview with reasoning effort ‘low’ (image by author)

Observations & Final Thoughts

Even the smaller models, such as gemini-3.1-flash-lite-previewand gpt-5.4-mini, with reasoning_effort set to low, extracted the messy purchase order accurately. It was all about using the right prompts.

It should be noted that this parsing is useful where accuracy is more important than speed, and a fast or real-time response is not required. For instance, in purchase order processing, accurate parsing for accurate price calculation is more important than speed.

Increasing the reasoning effort drastically increases the processing time and the cost. Bigger models, such as gpt-5.4 with reasoning effort set to high, become awfully slow. Therefore, a “high” reasoning, especially for bigger models, should be used only for documents with highly complex layouts.

With respect to cost and speed, gemini-3.1-flash-lite-preview stands out as the best choice.

The additional_instructions parameter can be used to supply use-case-specific parsing instructions. For instance, for legal documents, it can be set as “*Preserve footnote numbering exactly as it appears in the source*.”